

COURSE CODE/TITLE:	CPE 010 - Data Structures and Algorithms	SECTION:	CPE21S1
DATE PERFORMED:	August 18, 2026	DATE SUBMITTED:	August 22, 2026
NAME:	Benj Federic C. De Leon	INSTRUCTOR:	Engr. Jimlord M. Quejado

ACTIVITY NO. 6.1 Searching Techniques

1. PROCEDURE OUTPUT

ILO A: Create C++ code for searching data using different techniques

A.1 Sequential Search

Create a function called `linearSearch` that accepts necessary parameters to execute the algorithm described above.

B.1 Binary Search

Create a function called `binarySearch` that accepts necessary parameters to execute the algorithm described above.

main.cpp

```
#include <iostream>
#include <cstdlib> //for generating random integers
#include <time.h> //will be used for our seeding function
#include "node.h"
#include "searchingAlgo.h"

const int max_size = 50;

int main() {

    //generate random values:
    int dataset[max_size];

    srand(time(0));

    for(int i = 0; i < max_size; i++){
        dataset[i] = rand();
    }
```

```
for(int i = 0; i < max_size; i++){
    std::cout << dataset[i] << " ";
}

std::cout << std::endl;

//create a user input to ask what value to find:
int toFind;

std::cout << "what value you want to search? ";
std::cin >> toFind;

linearSearchArr(dataset, max_size, toFind);

//linear search:
std::cout << "what value you want to search? ";
std::cin >> toFind;

linearSearchArr(dataset, max_size, toFind);

//linked list: D I J L M O R

Node<char>* name1 = new_node('D');
Node<char>* name2 = new_node('I');
Node<char>* name3 = new_node('J');
Node<char>* name4 = new_node('L');
Node<char>* name5 = new_node('M');
Node<char>* name6 = new_node('O');
Node<char>* name7 = new_node('R');

name1->next = name2;
name2->next = name3;
name3->next = name4;
```

```
name4->next = name5;
name5->next = name6;
name6->next = name7;

std::cout << "Linear Search for Array 'D': ";
linearSearchArr(name1, 'D');

std::cout << "Linear Search for Linked List result: 'O' ";
linearSearchLL(name1, 'O');

//Binary Search for Linked List

std::cout << "\nLinked List Binary Search for 'R': ";
binarySearchLL(name1, 'R');

std::cout << "Linked List Binary Search for 'Z': ";
binarySearchLL(name1, 'Z');

//Binary Search for Array
int numbers[7] = {10, 20, 30, 40, 50, 60, 70};

std::cout << "\nBinary Search for Array result: ";
binarySearchArr(numbers, 7, 50);

std::cout << "Binary Search for Array result: ";
binarySearchArr(numbers, 7, 90);

return 0;
}
```

```
#include "node.h"

#ifndef SEARCHINGALGO_H
#define SEARCHINGALGO_H

template <typename T>
void linearSearchArr(T data[], int arrSize, T dataFind){
    //Step 1: I := 0
    int i = 0;

    //Repeat while I <= n
    while(i < arrSize){
        if(dataFind == data[i]){
            std::cout << "the search is success\n";
            return;
        }

        //increment i to move to the next element
        i++;
    }

    std::cout << "the element is not found\n";
    return;
}

template <typename T>
void linearSearchArr(Node<T>* node, T dataFind){
    while(node != nullptr){
        if(dataFind == node->data){
            std::cout << "the search is success\n";
            return;
        }
        node = node->next;
    }

    std::cout << "the element is not found\n";
```

```

    return;
}

template <typename T>
void linearSearchLL(Node<T>* node, T dataFind){
    //repeat the loop until the current node is not null
    while(node != nullptr){

        //if item = current node's data then display searching is
        successful
        if(dataFind == node->data){
            std::cout << "the search was a success\n";
            return;
        }

        //if not, move to the next node
        node = node->next;
    }

    //the searching reached nullptr -> search is unsuccessful
    std::cout << "the search was unsuccessful\n";
}

/*
getMiddle, must take the first and last node as its parameters.

1. Traverse the singly linked list using two pointers.
2. Move one pointer by one step ahead and the other pointer by two
   steps.
3. When the fast pointer reaches the end of the singly linked list,
   the slow pointer will reach the middle of the singly linked list.
4. Return slow pointer address.
*/

template <typename T>

```

```
Node<T>* getMiddle(Node<T>* start, Node<T>* last){

    //use two pointers:
    //slow moves one node at a time
    //fast moves two nodes at a time
    Node<T>* slow = start;
    Node<T>* fast = start;

    //get middle:
    while(fast != last && fast->next != last){

        //move slow (one step)
        slow = slow->next;

        //move fast (two steps)
        fast = fast->next->next;
    }

    return slow;
}
```

```
/*
Binary Search for Linked List
```

1. Start node is set to head of the list and the last node is set to NULL.
2. Middle element is calculated using the two pointers approach.
3. If the middle element is same as the key to be searched, we return it.
4. Else if middle element is greater than the key to be searched, search the LEFT side.
5. Else if middle element is less than the key to be searched, search the RIGHT side.
6. If the key is found or the entire linked list gets traversed, we stop the loop.

```
*/
```

```
template <typename T>
void binarySearchLL(Node<T>* start, T findData){

    Node<T>* last = nullptr;

    while(last == nullptr || last != start){

        //Find middle element using two pointers
        Node<T>* mid = getMiddle(start, last);

        //If middle element is empty/null, break loop
        if(mid == nullptr){
            break;
        }

        //If middle element is equal to key
        if(mid->data == findData){
            std::cout << "the search was a success\n";
            return;
        }

        //If middle element is greater than key,
        //search the LEFT side
        else if(mid->data > findData){
            last = mid;
        }

        //If middle element is less than key,
        //search the RIGHT side
        else{
            start = mid->next;
        }
    }

    //Search unsuccessful
    std::cout << "the search was unsuccessful\n";
}
```

```
}

// create a binary search using array : getmiddle and / binarySearchArr
template <typename T>
int getMiddle(T data[], int start, int last){

    //find the middle index
    int middle = (start + last) / 2;

    return middle;
}

template <typename T>
void binarySearchArr(T data[], int arrSize, T findData){

    //start of the array
    int start = 0;

    //last position of the array
    int last = arrSize - 1;

    while(start <= last){

        //Find middle element
        int mid = getMiddle(data, start, last);

        //If middle element is equal to key
        if(data[mid] == findData){
            std::cout << "the search was a success\n";
            return;
        }

        //If middle element is greater than key,
        //search the LEFT side
        else if(data[mid] > findData){
```



```
        last = mid - 1;
    }

    //If middle element is less than key,
    //search the RIGHT side
    else{
        start = mid + 1;
    }
}

//Search unsuccessful
std::cout << "the search was unsuccessful\n";
}

#endif
```

node.h

```
#ifndef NODE_H
#define NODE_H

template <typename T>
struct Node {
    T data;
    Node<T>* next;
};

template <typename T>
Node<T>* new_node(T data) {
    Node<T>* node = new Node<T>;
    node->data = data;
    node->next = nullptr;
    return node;
}
```

```
#endif
```

OUTPUT

```
9208 3045 13272 1916 10756 10820 29034 1529 1334 8178 22742 23435 12603 9458 233 18270 9693 29176 8018 18678 17122 3266 29606 28448 13105
11947 25511 4905 2871 31102 11461 21581 9865 28296 20197 26482 2251 23537 5218 3863 17872 13388 23377 20359 13570 1132 1034 13138 29701 31
609
what value you want to search? 1034
the search is success
what value you want to search? 1224
the element is not found
Linear Search for Array 'D': the search is success
Linear Search for Linked List result: '0' the search was a success

Linked List Binary Search for 'R': the search was a success
Linked List Binary Search for 'Z': the search was unsuccessful

Binary Search for Array result: the search was a success
Binary Search for Array result: the search was unsuccessful
PS C:\Users\Benj_Federic_De_Leon\Documents\CPE21S1\CPE_010-CPE21S1\HoA6p1>
```

Code analysis

This section tests two ways to find items in lists: linear search and binary search. Linear search looks at items one by one from start to finish, which works fine even if the list is messy. Binary search is much faster because it keeps splitting a sorted list in half to find the target. Binary search works quickly on simple arrays because you can jump straight to any slot, but it takes extra work on linked lists because you have to walk through nodes using two pointers just to find the middle.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO B: Solve different problems utilizing appropriate searching techniques in C++

Problem 1. Suppose you are doing a sequential search of the list [15, 18, 2, 19, 18, 0, 8, 14, 19, 14]. Utilizing both a linked list and an array approach to the list, use sequential search and identify how many comparisons would be necessary to find the key '18'?

Problem 2. Modify your sequential search algorithm so that it returns the count of repeating instances for a given search element 'k'. Test on the same list given in problem 1.

Problem 3. Suppose you have the following sorted list [3, 5, 6, 8, 11, 12, 14, 15, 17, 18] and are using the binary search algorithm. If you wanted to find the key 8, draw a diagram that shows how the searching works per iteration of the algorithm. Prove that your drawing is correct by implementing the algorithm and showing a screenshot of the code and the output console.

Problem 4. Modify the binary search algorithm so that the algorithm becomes recursive. Using this new

recursive binary search, implement a solution to the same problem for problem 3.

supplementary.cpp

```
#include <iostream>
#include <vector>
#include <list>
#include "supplementaryAlgo.h"

int main() {

    //Problem 1
    std::vector<int> arrList = {15, 18, 2, 19, 18, 0, 8, 14, 19, 14};
    std::list<int> llList = {15, 18, 2, 19, 18, 0, 8, 14, 19, 14};

    std::cout << "Problem 1:\n";

    std::cout << "Array approach: ";
    linearSearchCountArr(arrList, 18);

    std::cout << "\nLinked List approach: ";
    linearSearchCountLL(llList, 18);

    //Problem 2
    std::cout << "\nProblem 2:\n";
    countOccurrences(arrList, 18);

    //Problem 3:
    std::vector<int> sortedList = {3, 5, 6, 8, 11, 12, 14, 15, 17, 18};

    std::cout << "\nProblem 3:\n";
    binarySearchVerbose(sortedList, 8);

    //Problem 4:
```

```
std::cout << "\nProblem 4:\n";  
binarySearchRecursive(sortedList, 0, (int)sortedList.size() - 1,  
8);  
  
return 0;  
}
```

supplementary.h

```
#include <iostream>  
#include <vector>  
#include <list>  
  
#ifndef SUPPLEMENTARYALGO_H  
#define SUPPLEMENTARYALGO_H  
  
//Problem 1  
  
template <typename T>  
int linearSearchCountArr(std::vector<T>& data, T dataFind){  
    //Step 1: I := 0  
    int i = 0;  
    int comparisons = 0;  
  
    //Repeat while  
    while(i < (int)data.size()){  
        comparisons++;  
  
        if(dataFind == data[i]){  
            std::cout << "the search is success";  
            return comparisons;  
        }  
  
        i++;  
    }  
}
```

```
}

std::cout << "the element is not found\n";
return comparisons;
}

//Linked list version
template <typename T>
int linearSearchCountLL(std::list<T>& data, T dataFind){
    int comparisons = 0;
    int index = 0;

    //repeat the loop
    for(auto it = data.begin(); it != data.end(); ++it, ++index){
        comparisons++;

        if(dataFind == *it){
            std::cout << "the search was a success\n";
            return comparisons;
        }
    }

    //the searching reached the end -> search is unsuccessful
    std::cout << "the search was unsuccessful\n";
    return comparisons;
}

//Problem 2:

template <typename T>
int countOccurrences(std::vector<T>& data, T k){
    int count = 0;

    //Repeat while I <= n
    for(int i = 0; i < (int)data.size(); i++){
        if(data[i] == k){
            count++;
        }
    }
}
```

```
    }  
}  
  
    std::cout << "'" << k << "' appears " << count << " time(s) in the  
list\n";  
    return count;  
}  
  
//Problem 3:  
  
template <typename T>  
void binarySearchVerbose(std::vector<T>& data, T findData){  
  
    //start of the array  
    int start = 0;  
  
    //last position of the array  
    int last = (int)data.size() - 1;  
    int step = 1;  
  
    while(start <= last){  
  
        //Find middle element  
        int mid = (start + last) / 2;  
  
        std::cout << "  Iteration " << step << ": low = " << start  
                << ", mid = " << mid << " (value " << data[mid]  
                << "), high = " << last << "\n";  
        step++;  
  
        //If middle element is equal to key  
        if(data[mid] == findData){  
            std::cout << "the search was a success\n";  
            return;  
        }  
  
        else if(data[mid] > findData){
```

```
        last = mid - 1;
    }

    else{
        start = mid + 1;
    }
}

//Search unsuccessful
std::cout << "the search was unsuccessful\n";
}

//Problem 4:

template <typename T>
void binarySearchRecursive(std::vector<T>& data, int low, int high, T
findData, int step = 1){

    if(low > high){
        std::cout << "the search was unsuccessful\n";
        return;
    }

    int mid = (low + high) / 2;

    std::cout << "  Iteration " << step << ": low = " << low
        << ", mid = " << mid << " (value " << data[mid]
        << "), high = " << high << "\n";

    if(data[mid] == findData){
        std::cout << "the search was a success\n";
        return;
    }

    else if(data[mid] > findData){
```

```
        binarySearchRecursive(data, low, mid - 1, findData, step + 1);  
    }  
  
    else{  
        binarySearchRecursive(data, mid + 1, high, findData, step + 1);  
    }  
}  
  
#endif
```

OUTPUT

Problem 1:

Array approach: the search is success

Linked List approach: the search was a success

Problem 2:

'18' appears 2 time(s) in the list

Problem 3:

Iteration 1: low = 0, mid = 4 (value 11), high = 9

Iteration 2: low = 0, mid = 1 (value 5), high = 3

Iteration 3: low = 2, mid = 2 (value 6), high = 3

Iteration 4: low = 3, mid = 3 (value 8), high = 3

the search was a success

Problem 4:

Iteration 1: low = 0, mid = 4 (value 11), high = 9

Iteration 2: low = 0, mid = 1 (value 5), high = 3

Iteration 3: low = 2, mid = 2 (value 6), high = 3

Iteration 4: low = 3, mid = 3 (value 8), high = 3

the search was a success

PS C:\Users\Benj Federic De Leon\Documents\CPE21S1\CPE 010-CPE21S1\HoA6p1>

Code analysis

This section looks at how many steps search code takes to run. Finding the number 18 in the list takes **2 checks** for both arrays and linked lists because 18 is the second item. Checking every item in the list shows that 18 appears **2 times**. Finding the number 8 in a sorted list takes **4 steps**, checking index 4, then index 1, then index 2, and finally finding it at index 3. The recursive version follows these exact same 4 steps, but it uses a function calling itself instead of a standard loop.

Conclusion

In this lab, I learned that linear search is simple and works on unsorted lists, while binary search is much faster but requires sorted data. I also saw that binary search works best with arrays, whereas linked lists need extra steps to find the middle element.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: Claude

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: Code assistance, explanation of concepts, and summarization

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: Claude was used to find/fix some bugs and guide me through the supplementary code, but I reviewed and understood it.

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.